

PE6201 · Class 1 · Capsule 2 — Live Demo

Same task. Both ways.

Emerging AI Technologies · MSc in Enterprise Artificial Intelligence · NTU

Sort customer reviews into positive and negative — once with a **trained classifier**, once with a **foundation-model API**.

Cells 1–4 need no API key. Cell 5 onward calls a real model, and degrades gracefully if it can't.

Run the cells in order. Later cells depend on variables defined earlier.

Runtime: about 3 minutes.

CELL 1 — Setup and build the dataset

12,000 e-commerce reviews with mixed signals and 4% label noise — because real datasets are messy.

Nothing to say while this runs. Takes about two seconds.

```
In [1]: # 12,000 e-commerce reviews. Mixed signals, and 4% are mislabelled – because
# real datasets are.

import random, time, json, urllib.request
from IPython.display import HTML, display
random.seed(7)

DELIVERY_GOOD = ["arrived two days early", "shipping was quick", "delivery to Singapore was fast",
                  "packaging was solid", "came well wrapped"]
DELIVERY_BAD  = ["arrived a week late", "shipping took forever", "packaging was crushed",
                  "box was damaged", "delivery was a mess"]
PRODUCT_GOOD  = ["the material feels premium", "it works exactly as described", "better than I expected",
                  "the fit is perfect", "build quality is excellent", "my mother loves it"]
```

```

PRODUCT_BAD    = ["the material feels cheap","it stopped working after two uses","nothing like the photos",
                  "the fit is wrong","build quality is poor","it broke within a fortnight"]
SERVICE_GOOD  = ["seller replied within the hour","refund was processed immediately","very helpful support"]
SERVICE_BAD    = ["seller never replied","still waiting on a refund","support was useless"]
NEUTRAL         = ["ordered during the 11.11 sale","comes in a plain box","instructions are in three languages",
                  "bought this for my flat","colour is as shown","this is my second order","paid with PayNow"]
CLOSE_P        = ["Would buy again.","Recommend.","Happy overall.","Four stars.","Worth it."]
CLOSE_N        = ["Would not buy again.","Do not recommend.","Disappointed overall.","Two stars.","Not worth it."]

def make_review(label):
    if label == 1:
        good = random.sample(DELIVERY_GOOD+PRODUCT_GOOD+SERVICE_GOOD, k=random.randint(1,3))
        bad  = random.sample(DELIVERY_BAD +PRODUCT_BAD +SERVICE_BAD, k=random.randint(0,2))
    else:
        bad  = random.sample(DELIVERY_BAD +PRODUCT_BAD +SERVICE_BAD, k=random.randint(1,3))
        good = random.sample(DELIVERY_GOOD+PRODUCT_GOOD+SERVICE_GOOD, k=random.randint(0,2))
    bits = good + bad
    random.shuffle(bits)
    if random.random() < 0.6:
        bits.insert(random.randint(0, len(bits)), random.choice(NEUTRAL))
    txt = ", ".join(bits).capitalize() + ". "
    if random.random() < 0.55:
        txt += random.choice(CLOSE_P if label == 1 else CLOSE_N)
    return txt.strip()

X, y = [], []
for _ in range(12000):
    lab = random.randint(0, 1)
    X.append(make_review(lab))
    y.append(1 - lab if random.random() < 0.04 else lab)      # 4% label noise

X_train, y_train, X_test, y_test = X[:-2000], y[:-2000], X[-2000:], y[-2000:]
print(f"{len(X):,} reviews      {len(X_train):,} for training      {len(X_test):,} held back for testing")

```

12,000 reviews · 10,000 for training · 2,000 held back for testing

CELL 2 — Look at the data

Ask the room first: *how hard does this look?*

Note that most reviews contain **both** good and bad points. The label is the overall verdict, not a keyword count — which is why this is a real classification problem and not a word search.

In [2]: *# Ask the room: how hard does this look?*

```
def card(txt, lab):
    c = "#1F6B45" if lab == 1 else "#D71440"
    tag = "POSITIVE" if lab == 1 else "NEGATIVE"
    return (f'<div style="font-family:Arial;margin:8px 0;padding:12px 16px;'
            f'border-left:5px solid {c};background:#F6F6F8;">'
            f'<span style="color:{c};font-weight:700;font-size:12px;'
            f'letter-spacing:1px;">{tag}</span><br>'
            f'<span style="font-size:17px;color:#1a1a1a;">{txt}</span></div>')

display(HTML('<div style="font-family:Arial;font-size:15px;color:#5A5A5A;">'
            'Sample a few reviews. The label is the <i>overall</i> verdict – note that most contain '
            'both good and bad points.</div>'
            + """.join(card(X[i], y[i]) for i in range(20)))
```

Sample a few reviews. The label is the *overall* verdict — note that most contain both good and bad points.

POSITIVE

Colour is as shown, arrived a week late, it works exactly as described, shipping took forever. Would buy again.

NEGATIVE

Arrived two days early, it works exactly as described, shipping took forever.

NEGATIVE

It stopped working after two uses, colour is as shown, arrived a week late, build quality is poor. Disappointed overall.

NEGATIVE

Colour is as shown, build quality is poor, my mother loves it, packaging was solid. Do not recommend.

POSITIVE

The material feels cheap, better than i expected, build quality is excellent.

POSITIVE

Seller replied within the hour, the material feels premium, comes in a plain box, delivery was a mess, better than i expected.

NEGATIVE

Seller replied within the hour, build quality is poor, the material feels cheap, ordered during the 11.11 sale, still waiting on a refund.

POSITIVE

My mother loves it, arrived two days early, shipping was quick, seller never replied, delivery was a mess. Four stars.

NEGATIVE

Arrived a week late, delivery to singapore was fast, box was damaged, ordered during the 11.11 sale. Two stars.

NEGATIVE

Seller replied within the hour, the fit is wrong, support was useless. Do not recommend.

NEGATIVE

Comes in a plain box, box was damaged, packaging was solid, arrived two days early. Disappointed overall.

POSITIVE

Build quality is poor, seller replied within the hour, it broke within a fortnight, bought this for my flat. Would buy again.

POSITIVE

Packaging was solid. Would buy again.

NEGATIVE

Arrived a week late, the fit is perfect, delivery to singapore was fast. Do not recommend.

NEGATIVE

The material feels cheap, delivery was a mess, build quality is poor, better than i expected. Two stars.

NEGATIVE

Seller replied within the hour, seller never replied, comes in a plain box. Do not recommend.

POSITIVE

Seller replied within the hour, arrived a week late, still waiting on a refund.

POSITIVE

The material feels premium, delivery to singapore was fast, very helpful support.

NEGATIVE

Support was useless, instructions are in three languages, seller replied within the hour.

POSITIVE

Came well wrapped, packaging was solid, build quality is poor, the material feels cheap. Recommend.

CELL 3 — Approach A — with 10 labelled examples

A trained classifier, given almost nothing to learn from.

Let the 52% sit for a beat. A coin flip is 50%.

```
In [3]: import time # time to train

import sklearn
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
```

```

from sklearn.metrics import accuracy_score

def train_classifier(n):
    m = make_pipeline(TfidfVectorizer(ngram_range=(1,2), min_df=1),
                       LogisticRegression(max_iter=1000))
    t0 = time.time(); m.fit(X_train[:n], y_train[:n]); fit_s = time.time() - t0
    t0 = time.time(); pred = m.predict(X_test); pred_ms = (time.time()-t0)/len(X_test)*1000
    return accuracy_score(y_test, pred), fit_s, pred_ms, m

acc10, fit10, ms10, _ = train_classifier(10)
print(f"Trained on 10 examples -> accuracy {acc10*100:.1f}%")
print(f"Time taken to train:      {fit10:.4f} seconds")
print(f"Time taken to predict:    {ms10:.4f} ms per sample")
print("\nA coin flip is 50%.")

```

Trained on 10 examples -> accuracy 52.9%
Time taken to train: 0.0137 seconds
Time taken to predict: 0.0083 ms per sample

A coin flip is 50%.

CELL 4 — Approach A — with 10,000

This is the moment.

Same code, same model, same test set. **Only the amount of labelled data changed.**

Say it out loud: somebody had to read and label every one of those ten thousand reviews.

In [4]: *# THE MOMENT. Same code. Same model. Only the amount of labelled data changed.*

```

curve = []
for n in (10, 50, 200, 1000, 10000):
    a, f, ms, model = train_classifier(n)
    curve.append((n, a, f, ms))

rows = ""
for n, a, f, ms in curve:
    w = int(a * 380)

```

```

rows += (f'<tr>'
        f'<td style="padding:7px 14px;font-family:Menlo,monospace;font-size:17px;'
        f'text-align:right;color:#5A5A5A;">{n:,}</td>'
        f'<td style="padding:7px 14px;">'
        f'<div style="display:inline-block;height:24px;width:{w}px;'
        f'background:#00FF00;border-radius:3px;"></div></td>'
        f'<td style="padding:7px 14px;font-family:Arial;font-size:19px;'
        f'font-weight:700;color:#00ff00;">{a*100:.1f}%</td></tr>')

display(HTML('<div style="font-family:Arial;font-size:15px;color:#ffffff;margin-bottom:6px;">'
            'Accuracy vs. number of labelled training examples</div>'
            f'<table style="border-collapse:collapse;">{rows}</table>'
            '<div style="font-family:Arial;font-size:17px;margin-top:14px;max-width:780px;">'
            'Nothing about the model changed. <b>Only the amount of labelled data.</b> '
            'Somebody had to read and label every one of those 10,000 reviews.</div>'))

acc_clf, fit_clf, ms_clf, clf = curve[-1][1], curve[-1][2], curve[-1][3], model
print(f"\nBest classifier: {acc_clf*100:.1f}% | trained in {fit_clf:.2f}s | {ms_clf:.3f} ms per prediction")

```

10		52.9%
50		67.9%
200		77.3%
1,000		85.2%
10,000		87.2%

Nothing about the model changed. **Only the amount of labelled data.** Somebody had to read and label every one of those 10,000 reviews.

Best classifier: 87.2% | trained in 0.11s | 0.010 ms per prediction

CELL 5 — Approach B — zero examples

A foundation model that has never seen your data, your labels, or your task.

In class: set `RUN_LIVE = True` and paste your OpenRouter key (Students get their OpenRouter key post class one). **Student note:** 50 calls costs a fraction of a cent.

If it fails: the fallback prints representative numbers and the demo carries on. Nobody will know.

```
In [6]: # A foundation model that has never seen your data. Set RUN_LIVE = True in class.

RUN_LIVE = True
API_KEY = "" # OpenRouter key
MODEL = "openai/gpt-4o-mini"
N_SAMPLE = 50 # keep the cost trivial

import os
from dotenv import load_dotenv

# Force-load the .env file from the current directory
load_dotenv(override=True)

API_KEY = os.environ.get("MY_PRIVATE_OPENROUTER_KEY") # os.environ.get("OPENROUTER_API_KEY").. my private open rout
#print(API_KEY)

PROMPT = ("Is this product review positive or negative overall? "
          "Reply with exactly one word: positive or negative.\n\nReview: ")

def ask_model(review):
    req = urllib.request.Request(
        "https://openrouter.ai/api/v1/chat/completions",
        data=json.dumps({"model": MODEL, "max_tokens": 4,
                        "messages": [{"role": "user", "content": PROMPT + review}]}).encode(),
        headers={"Authorization": f"Bearer {API_KEY}", "Content-Type": "application/json"})
    with urllib.request.urlopen(req, timeout=25) as r:
        return json.load(r)["choices"][0]["message"]["content"].strip().lower()
#print(RUN_LIVE)
#print(API_KEY)
```

```

if RUN_LIVE and API_KEY:
    correct, t0 = 0, time.time()
    for i in range(N_SAMPLE):
        try:
            ans = ask_model(X_test[i])
            if ("positive" in ans) == (y_test[i] == 1):
                correct += 1
        except Exception as e:
            print("call failed:", e); break
    elapsed = time.time() - t0
    acc_api = correct / N_SAMPLE
    ms_api = elapsed / N_SAMPLE * 1000
    print(f"Foundation model, ZERO training examples -> {acc_api*100:.1f}% "
          f"on {N_SAMPLE} reviews | {ms_api:.0f} ms per prediction")
else:
    acc_api, ms_api = 0.88, 850.0 # representative fallback
    print(f"[fallback - no live call] roughly {acc_api*100:.0f}%, about {ms_api:.0f} ms per prediction")
    print("Set RUN_LIVE = True and paste a key to run it for real.")

```

Foundation model, ZERO training examples -> 84.0% on 50 reviews | 1187 ms per prediction

CELL 6 — Side by side

The comparison table from the slide, filled in with your own numbers.

The line to land: *neither is better — one needs your data, the other needs your money.*

```

In [7]: comp = [("Labelled examples needed", "10,000", "0"),
                ("Accuracy", f"{acc_clf*100:.1f}%", f"~{acc_api*100:.0f}%"),
                ("Time to first result", "hours of labelling", "about 30 seconds"),
                ("Speed per prediction", f"{ms_clf:.3f} ms", f"{ms_api:.0f} ms"),
                ("Cost per 1,000", "$0.00", "real money, every time"),
                ("Change the task", "relabel and retrain", "rewrite one sentence")]

body = "".join(
    f'<tr><td style="padding:9px 16px;font-family:Arial;font-size:16px;color:#5A5A5A;">{a}</td>'
    f'<td style="padding:9px 16px;font-family:Arial;font-size:16px;font-weight:600;'
    f'color:#00ff00;">{b}</td>'
    f'<td style="padding:9px 16px;font-family:Arial;font-size:16px;font-weight:600;'
    f'color:#D71440;">{c}</td></tr>' for a,b,c in comp)

```

```
display(HTML(
  '<table style="border-collapse:collapse;margin-top:10px;">'
  '<tr style="background:#181C62;color:#fff;font-family:Arial;font-size:15px;">'
  '<th style="padding:10px 16px;text-align:left;"></th>'
  '<th style="padding:10px 16px;text-align:left;">Trained classifier</th>'
  '<th style="padding:10px 16px;text-align:left;">Foundation model</th></tr>'
  f'{{body}}</table>'
  '<div style="font-family:Arial;font-size:18px;margin-top:16px;max-width:800px;">'
  'Neither is better. <b>One needs your data. The other needs your money.</b></div>'))
```

	Trained classifier	Foundation model
Labelled examples needed	10,000	0
Accuracy	87.2%	~84%
Time to first result	hours of labelling	about 30 seconds
Speed per prediction	0.010 ms	1187 ms
Cost per 1,000	~\$0.00	real money, every time
Change the task	relabel and retrain	rewrite one sentence

Neither is better. **One needs your data. The other needs your money.**

CELL 7 — Now change the task

The row that explains everything.

The foundation model does a brand-new task with one sentence rewritten. The classifier needs ten thousand reviews relabelled first — **28 hours of human work** before a single line of code runs.

That collapsed loop is why generative AI spread faster than the PC. Not because it is smarter.

In [8]: *# The row that actually explains why generative AI spread so fast.*

```
NEW_PROMPT = ("Does this review mention DELIVERY or SHIPPING? "
              "Reply with exactly one word: yes or no.\n\nReview: ")

print("New question: 'Does this review mention delivery?'\n")
print("FOUNDATION MODEL – one sentence rewritten. Working now:")
print(f"    {NEW_PROMPT.strip()[:100]}...\n")
print("CLASSIFIER – needs 10,000 reviews relabelled for the new question.")
print(f"    At 10 seconds per review that is {10000*10/3600:.0f} hours of human work")
print("    before a single line of code can run.\n")
print("Same model. Same data. The difference is the length of the loop.")

if RUN_LIVE and API_KEY:
    for i in range(5):
        req = urllib.request.Request(
            "https://openrouter.ai/api/v1/chat/completions",
            data=json.dumps({"model": MODEL, "max_tokens": 4,
                            "messages": [{"role": "user", "content": NEW_PROMPT + X_test[i]}]}).encode(),
            headers={"Authorization": f"Bearer {API_KEY}", "Content-Type": "application/json"})
        with urllib.request.urlopen(req, timeout=25) as r:
            ans = json.load(r)["choices"][0]["message"]["content"].strip()
        print(f"    [{ans:>3}] {X_test[i][:200]}...")
```

New question: 'Does this review mention delivery?'

FOUNDATION MODEL – one sentence rewritten. Working now:

"Does this review mention DELIVERY or SHIPPING? Reply with exactly one word: yes or no.

Review:..."

CLASSIFIER – needs 10,000 reviews relabelled for the new question.

At 10 seconds per review that is 28 hours of human work
before a single line of code can run.

Same model. Same data. The difference is the length of the loop.

[no] The material feels premium. Happy overall....

[yes] Delivery was a mess....

[no] Packaging was solid, the fit is wrong, comes in a plain box, better than i expected, support was useless, the fit is perfect. Worth it....

[yes] Delivery was a mess, refund was processed immediately, comes in a plain box, seller replied within the hour, arrived two days early. Recommend....

[no] My mother loves it....